Best Fit

```
#include <stdio.h>

void implimentBestFit(int blockSize[], int blocks, int processSize[], int
proccesses)

{

    int allocation[proccesses];

    int occupied[blocks];

    for(int i = 0; i < proccesses; i++){

        allocation[i] = -1;

    }

    for(int i = 0; i < blocks; i++){

        occupied[i] = 0;

    }

    for (int i = 0; i < proccesses; i++)

    {

        int indexPlaced = -1;

        for (int j = 0; j < blocks; j++) {

            if (blockSize[j] >= processSize[i] && !occupied[j])

            {

                if (indexPlaced == -1)

                    indexPlaced = j;


                else if (blockSize[j] < blockSize[indexPlaced])

                    indexPlaced = j;

            }

        }


        if (indexPlaced != -1)

        {

            allocation[i] = indexPlaced;


            occupied[indexPlaced] = 1;
```

```
        }
    }

    printf("\nProcess No.\tProcess Size\tBlock no.\n");
    for (int i = 0; i < proccesses; i++)
    {
        printf("%d \t\t\t %d \t\t\t", i+1, processSize[i]);
        if (allocation[i] != -1)
            printf("%d\n",allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
}

int main()
{
    int blockSize[] = {100, 50, 30, 120, 35};
    int processSize[] = {40, 10, 30, 60};
    int blocks = sizeof(blockSize)/sizeof(blockSize[0]);
    int proccesses = sizeof(processSize)/sizeof(processSize[0]);
    implimentBestFit(blockSize, blocks, processSize, proccesses);

    return 0 ;
}
```

Output:

```
[Running] cd "g:\OS\lab7\" && gcc bestfit.c -o bestfit && "g:\OS\lab7\"bestfit

Process No. Process Size    Block no.
1            40             2
2            10             3
3            30             5
4            60             1

[Done] exited with code=0 in 1.008 seconds
```

First Fit

```
#include <stdio.h>

void implimentFirstFit(int blockSize[], int blocks, int processSize[],
int processes)

{

    int allocate[processes];

    int occupied[blocks];

    for(int i = 0; i < processes; i++)

     {

           allocate[i] = -1;

     }


     for(int i = 0; i < blocks; i++){

        occupied[i] = 0;

    }

    for (int i = 0; i < processes; i++)

    {

        for (int j = 0; j < blocks; j++)

        {

        if (!occupied[j] && blockSize[j] >= processSize[i])

            {

                // allocate block j to p[i] process

                allocate[i] = j;

                occupied[j] = 1;


                break;

            }

        }

    }


    printf("\nProcess No.\tProcess Size\tBlock no.\n");

    for (int i = 0; i < processes; i++)
```

```
    {
        printf("%d \t\t\t %d \t\t\t", i+1, processSize[i]);
        if (allocate[i] != -1)
            printf("%d\n",allocate[i] + 1);
        else
            printf("Not Allocated\n");
    }
}

void main()
{
    int blockSize[] = {30, 5, 10};
    int processSize[] = {10, 6, 9};
    int m = sizeof(blockSize)/sizeof(blockSize[0]);
    int n = sizeof(processSize)/sizeof(processSize[0]);

    implimentFirstFit(blockSize, m, processSize, n);
}
```

Output:

```
Process No. Process Size    Block no.
1           10              1
2           6           3
3           9           Not Allocated
```

Worst Fit

```
#include <stdio.h>

void implimentWorstFit(int blockSize[], int blocks, int processSize[], int processes)

{

    int allocation[processes];

    int occupied[blocks];


    for(int i = 0; i < processes; i++){

        allocation[i] = -1;

    }


    for(int i = 0; i < blocks; i++){

        occupied[i] = 0;

    }

    for (int i=0; i < processes; i++)

    {

      int indexPlaced = -1;

      for(int j = 0; j < blocks; j++)

      {

          if(blockSize[j] >= processSize[i] && !occupied[j])

             {

                 if (indexPlaced == -1)

                     indexPlaced = j;

                 else if (blockSize[indexPlaced] < blockSize[j])

                     indexPlaced = j;

             }

         }

         if (indexPlaced != -1)

         {

             // allocate this block j to process p[i]

             allocation[i] = indexPlaced;
```

```
            occupied[indexPlaced] = 1;
            blockSize[indexPlaced] -= processSize[i];
        }
    }

    printf("\nProcess No.\tProcess Size\tBlock no.\n");
    for (int i = 0; i < processes; i++)
    {
        printf("%d \t\t\t %d \t\t\t", i+1, processSize[i]);
        if (allocation[i] != -1)
            printf("%d\n",allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
}
int main()
{
    int blockSize[] = {100, 50, 30, 120, 35};
    int processSize[] = {40, 10, 30, 60};
    int blocks = sizeof(blockSize)/sizeof(blockSize[0]);
    int processes = sizeof(processSize)/sizeof(processSize[0]);
    implimentWorstFit(blockSize, blocks, processSize, processes);
    return 0;
}
```

Output

```
[Running] cd "g:\OS\lab7\" && gcc worstfit.c -o worstfit && "g:\OS\lab7\"worstfit

Process No. Process Size    Block no.
1           40              4
2           10              1
3           30              2
4           60              Not Allocated

[Done] exited with code=0 in 0.967 seconds
```